

Introduction to AI, Machine Learning and Data Science

From data to models

Tokey Tahmid

Version 1.0.1



Plan

Time	Session
11:30	Welcome and setup
11:35	Part 1: From data to models
11:57	<i>Practical 1:</i> the Keeling curve
12:22	Part 2: Classical ML with scikit-learn
12:40	<i>Practical 2:</i> classifying wines by their chemistry
13:00	Lunch break
14:00	Part 3: Neural networks, transformers and ML on HPC
14:22	<i>Practical 3:</i> a first neural network + a batch job
14:47	Wrap-up and questions

By the end you can

Plan

Time	Session
11:30	Welcome and setup
11:35	Part 1: From data to models
11:57	<i>Practical 1:</i> the Keeling curve
12:22	Part 2: Classical ML with scikit-learn
12:40	<i>Practical 2:</i> classifying wines by their chemistry
13:00	Lunch break
14:00	Part 3: Neural networks, transformers and ML on HPC
14:22	<i>Practical 3:</i> a first neural network + a batch job
14:47	Wrap-up and questions

By the end you can

- explain how AI, ML, deep learning and data science relate

Plan

Time	Session
11:30	Welcome and setup
11:35	Part 1: From data to models
11:57	<i>Practical 1:</i> the Keeling curve
12:22	Part 2: Classical ML with scikit-learn
12:40	<i>Practical 2:</i> classifying wines by their chemistry
13:00	Lunch break
14:00	Part 3: Neural networks, transformers and ML on HPC
14:22	<i>Practical 3:</i> a first neural network + a batch job
14:47	Wrap-up and questions

By the end you can

- explain how AI, ML, deep learning and data science relate
- describe any ML method as **data + model + loss + optimiser**

Plan

Time	Session
11:30	Welcome and setup
11:35	Part 1: From data to models
11:57	<i>Practical 1:</i> the Keeling curve
12:22	Part 2: Classical ML with scikit-learn
12:40	<i>Practical 2:</i> classifying wines by their chemistry
13:00	Lunch break
14:00	Part 3: Neural networks, transformers and ML on HPC
14:22	<i>Practical 3:</i> a first neural network + a batch job
14:47	Wrap-up and questions

By the end you can

- explain how AI, ML, deep learning and data science relate
- describe any ML method as **data + model + loss + optimiser**
- explore and clean a dataset with **pandas**

Plan

Time	Session
11:30	Welcome and setup
11:35	Part 1: From data to models
11:57	<i>Practical 1:</i> the Keeling curve
12:22	Part 2: Classical ML with scikit-learn
12:40	<i>Practical 2:</i> classifying wines by their chemistry
13:00	Lunch break
14:00	Part 3: Neural networks, transformers and ML on HPC
14:22	<i>Practical 3:</i> a first neural network + a batch job
14:47	Wrap-up and questions

By the end you can

- explain how AI, ML, deep learning and data science relate
- describe any ML method as **data + model + loss + optimiser**
- explore and clean a dataset with **pandas**
- train, compare and tune models with **scikit-learn**, without fooling yourself

Plan

Time	Session
11:30	Welcome and setup
11:35	Part 1: From data to models
11:57	<i>Practical 1:</i> the Keeling curve
12:22	Part 2: Classical ML with scikit-learn
12:40	<i>Practical 2:</i> classifying wines by their chemistry
13:00	Lunch break
14:00	Part 3: Neural networks, transformers and ML on HPC
14:22	<i>Practical 3:</i> a first neural network + a batch job
14:47	Wrap-up and questions

By the end you can

- explain how AI, ML, deep learning and data science relate
- describe any ML method as **data + model + loss + optimiser**
- explore and clean a dataset with **pandas**
- train, compare and tune models with **scikit-learn**, without fooling yourself
- explain how neural networks and transformers work, and why GPUs matter

Plan

Time	Session
11:30	Welcome and setup
11:35	Part 1: From data to models
11:57	<i>Practical 1:</i> the Keeling curve
12:22	Part 2: Classical ML with scikit-learn
12:40	<i>Practical 2:</i> classifying wines by their chemistry
13:00	Lunch break
14:00	Part 3: Neural networks, transformers and ML on HPC
14:22	<i>Practical 3:</i> a first neural network + a batch job
14:47	Wrap-up and questions

By the end you can

- explain how AI, ML, deep learning and data science relate
- describe any ML method as **data + model + loss + optimiser**
- explore and clean a dataset with **pandas**
- train, compare and tune models with **scikit-learn**, without fooling yourself
- explain how neural networks and transformers work, and why GPUs matter
- run an ML job on **MeluXina**

Setup: start your engines now



ON YOUR LAPTOP, THEN ON MELUXINA

```
# 1. Log in to MeluXina
ssh -p 8822 <username>@login.lxp.lu
# Or Shell Access with Open On Demand

# 2. Get the course material
git clone https://github.com/JakeTWilliams/decarb_oct_26_material.git
# Or if already cloned
git pull
cd day_2/notebooks/intro_to_ai_ml

# 3. Start JupyterLab in Open OnDemand
#   Apps > JupyterLab: p201580, gpu, 4 hours
```

Login node

Never run notebooks or training on the login node.

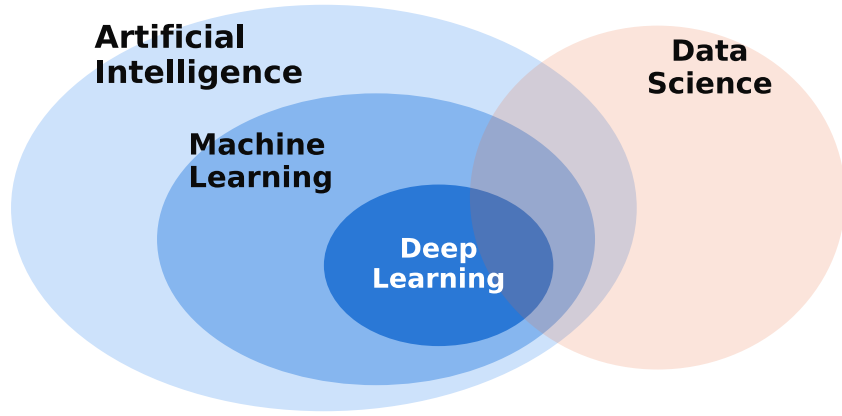
No MeluXina access?

Everything also runs on a laptop with NumPy, pandas, Matplotlib, scikit-learn and JupyterLab installed.

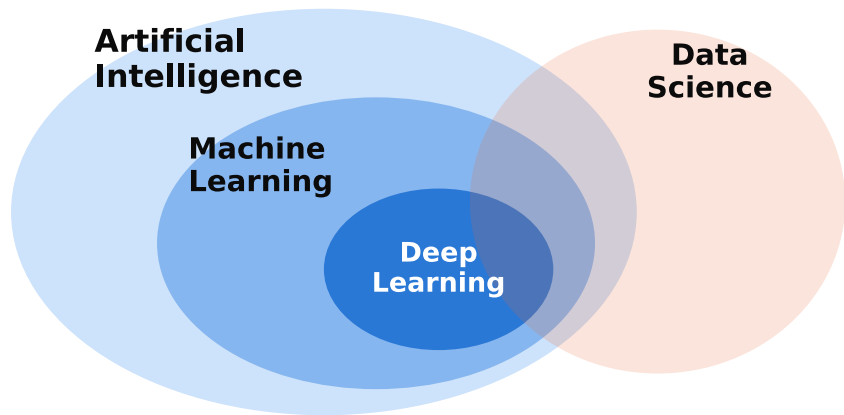
Part 1

From data to models

AI, machine learning, deep learning, data science

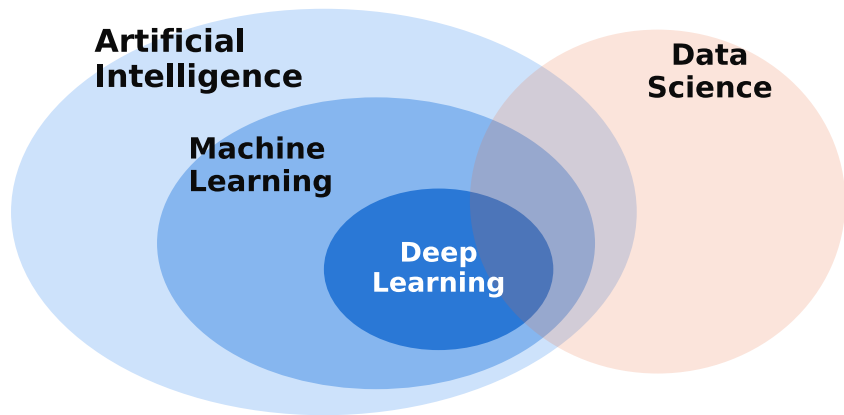


AI, machine learning, deep learning, data science



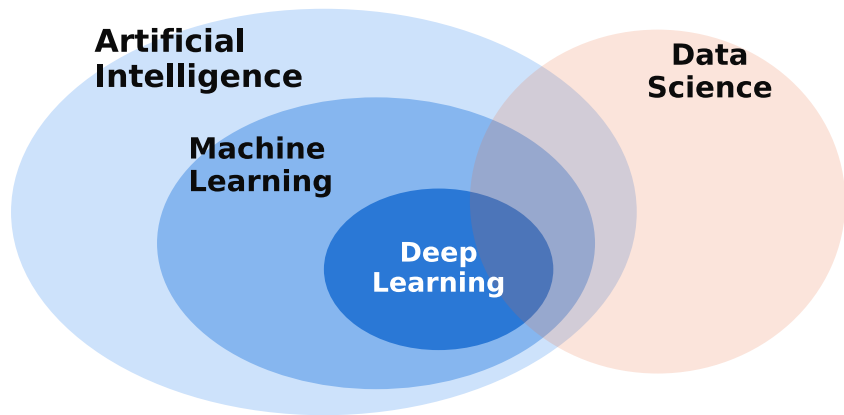
- **Artificial intelligence:** systems that perform tasks we associate with intelligence: perceiving, reasoning, deciding.

AI, machine learning, deep learning, data science



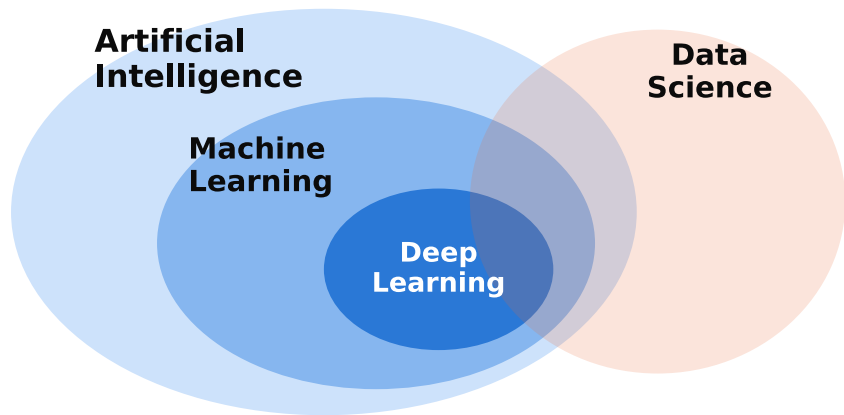
- **Artificial intelligence:** systems that perform tasks we associate with intelligence: perceiving, reasoning, deciding.
- **Machine learning:** the part of AI where behaviour is **learned from data** instead of programmed as rules.

AI, machine learning, deep learning, data science



- **Artificial intelligence:** systems that perform tasks we associate with intelligence: perceiving, reasoning, deciding.
- **Machine learning:** the part of AI where behaviour is **learned from data** instead of programmed as rules.
- **Deep learning:** ML with many-layered **neural networks** that learn their own features from raw data.

AI, machine learning, deep learning, data science



- **Artificial intelligence:** systems that perform tasks we associate with intelligence: perceiving, reasoning, deciding.
- **Machine learning:** the part of AI where behaviour is **learned from data** instead of programmed as rules.
- **Deep learning:** ML with many-layered **neural networks** that learn their own features from raw data.
- **Data science:** turning data into knowledge. Statistics + computing + domain expertise, with ML as one tool among many.

You already do machine learning

A fit you have probably done

Time a program at a few problem sizes N . The runtime grows roughly linearly, so fit a straight line by least squares:

$$t \approx w \cdot N + b$$

Then predict the runtime for a larger N , for example to choose the `--time` of your next Slurm job.

You already do machine learning

A fit you have probably done

Time a program at a few problem sizes N . The runtime grows roughly linearly, so fit a straight line by least squares:

$$t \approx w \cdot N + b$$

Then predict the runtime for a larger N , for example to choose the `--time` of your next Slurm job.

That is **supervised learning**: data, a model with parameters, a loss (squared error), an optimiser, and prediction on new inputs.

You already do machine learning

A fit you have probably done

Time a program at a few problem sizes N . The runtime grows roughly linearly, so fit a straight line by least squares:

$$t \approx w \cdot N + b$$

Then predict the runtime for a larger N , for example to choose the `--time` of your next Slurm job.

That is **supervised learning**: data, a model with parameters, a loss (squared error), an optimiser, and prediction on new inputs.

What machine learning changes

- **Many parameters**: thousands to billions, not two
- **Flexible models** instead of a known equation
- **Many inputs** (features) at once
- Focus on **predicting new data** more than on interpreting parameters

	Input	Output
Classical modelling	rules + data	answers
Machine learning	data + answers	rules

Where AI is used today

Language

Large language models (ChatGPT, Claude, Llama) are trained on thousands of GPUs for weeks: Llama 3.1 used over 16,000 GPUs.

Images

Image models recognise objects in photos, read medical scans and map land use from satellite images.

Weather forecasting

ML weather models (GraphCast, ECMWF's AIFS) give 10-day forecasts in minutes; AIFS is operational since 2025.

Surrogate models

Networks trained on simulation output approximate an expensive simulation in a fraction of the time.

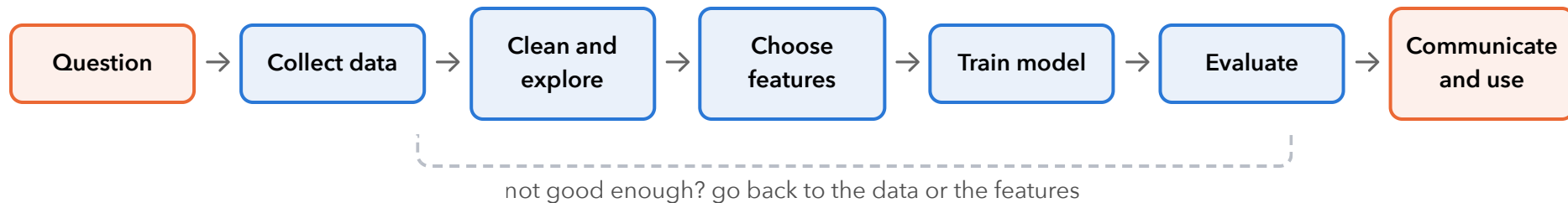
Research breakthroughs

AlphaFold predicts the 3D shape of proteins; it earned a share of the 2024 Nobel Prize in Chemistry.

Running data centres

ML predicts job runtimes, spots failing nodes and tunes cooling: up to 40 % less cooling energy at Google.

The data science workflow



- **The question comes first.** What decision or insight is the model for? What would count as success?
- **Most of the time goes into the first boxes:** collecting, cleaning and understanding data.
- **It is a loop, not a line.** Evaluation sends you back to the data or the features far more often than to a fancier model.
- **Garbage in, garbage out.** No algorithm fixes biased or broken data.

Data comes in shapes

Tabular data: the basic vocabulary

alcohol	flavanoids	proline	...	cultivar
14.2	3.06	1065	...	0
12.4	0.57	520	...	2
12.3	2.00	680	...	1

- a **row** is a *sample* (an observation)
- a **column** is a *feature* (an input variable)
- the last column is the **target** or **label**
- We write the features as a matrix $\mathbf{X}$ of shape $n \times d$ and the targets as a vector $\mathbf{y}$ of length n .

Other shapes, other models

- **Images and grids:** photos, scans, simulation fields
- **Sequences:** time series, text, audio, log files
- **Graphs:** networks of computers or people, meshes, molecules
- **Point clouds and sets:** 3D scans, particles

Much of deep learning is about model architectures that respect these shapes (Part 3).

Types of learning

Type	You have	Goal	Example
Supervised: regression	inputs + numeric targets	predict a number	a job's runtime from its size and node count
Supervised: classification	inputs + categories	predict a class	spam or not spam; which digit is in an image
Unsupervised: clustering	inputs only	find groups	group jobs by their CPU, memory and I/O profiles
Unsupervised: dimensionality reduction	inputs only	find low-dimensional structure	summarise hundreds of monitoring metrics in a few
Self-supervised	lots of raw data	predict hidden parts of the data itself	LLMs: predict the next word
Reinforcement learning	an environment + a reward	learn a strategy of actions	game playing; data-centre cooling control

Today we practise the first four. Self-supervised learning is how today's **foundation models** are trained (Part 3).

Quick check: which kind of learning?

1. Predict how many hours a simulation job will run from its input size and node count, using the logs of 5,000 past jobs.

2. Sort 10,000 news articles into topics. Nobody has labelled them.

3. Warn whether a compute node will fail in the next hour, trained on past monitoring data where failures were recorded.

Quick check: which kind of learning?

1. Predict how many hours a simulation job will run from its input size and node count, using the logs of 5,000 past jobs. supervised regression

2. Sort 10,000 news articles into topics. Nobody has labelled them.

3. Warn whether a compute node will fail in the next hour, trained on past monitoring data where failures were recorded.

Quick check: which kind of learning?

1. Predict how many hours a simulation job will run from its input size and node count, using the logs of 5,000 past jobs. supervised regression

2. Sort 10,000 news articles into topics. Nobody has labelled them. unsupervised clustering

3. Warn whether a compute node will fail in the next hour, trained on past monitoring data where failures were recorded.

Quick check: which kind of learning?

1. Predict how many hours a simulation job will run from its input size and node count, using the logs of 5,000 past jobs. supervised regression

2. Sort 10,000 news articles into topics. Nobody has labelled them. unsupervised clustering

3. Warn whether a compute node will fail in the next hour, trained on past monitoring data where failures were recorded. supervised classification (beware: failures are rare)

The recipe behind (almost) every ML method

1. Data

pairs (x_i, y_i)
for $i = 1, \dots, n$

2. Model

a function $f_\theta(x)$
with parameters θ

3. Loss

$L(\theta)$ measures
how wrong f_θ is

4. Optimiser

an algorithm that
changes θ to reduce L

$$\theta^* = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n \ell(f_\theta(x_i), y_i)$$

Linear regression, random forests, and the transformers behind ChatGPT all fit this template. Only the model, the loss and the optimiser change. Training a neural network uses exactly these four ingredients.

The recipe in miniature: linear regression

Model: $\hat{y} = wx + b$ **Loss** (mean squared error):

$$L(w, b) = \frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$$

Gradients (how the loss changes with each parameter):

$$\frac{\partial L}{\partial w} = \frac{2}{n} \sum_i (\hat{y}_i - y_i) x_i$$

$$\frac{\partial L}{\partial b} = \frac{2}{n} \sum_i (\hat{y}_i - y_i)$$

Optimiser (gradient descent, learning rate η):

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

```
PRACTICAL 1 (NUMPY)

w, b = 0.0, 0.0
for step in range(100):
    error = (w * x + b) - y          # model
    dw = 2 * np.mean(error * x)      # gradients
    db = 2 * np.mean(error)          # of the loss
    w -= lr * dw                     # gradient
    b -= lr * db                     # descent step
```

Linear regression has a closed-form solution. We use gradient descent anyway, because it is how models with millions of parameters are trained.

The recipe in miniature: linear regression

Model: $\hat{y} = wx + b$ **Loss** (mean squared error):

$$L(w, b) = \frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$$

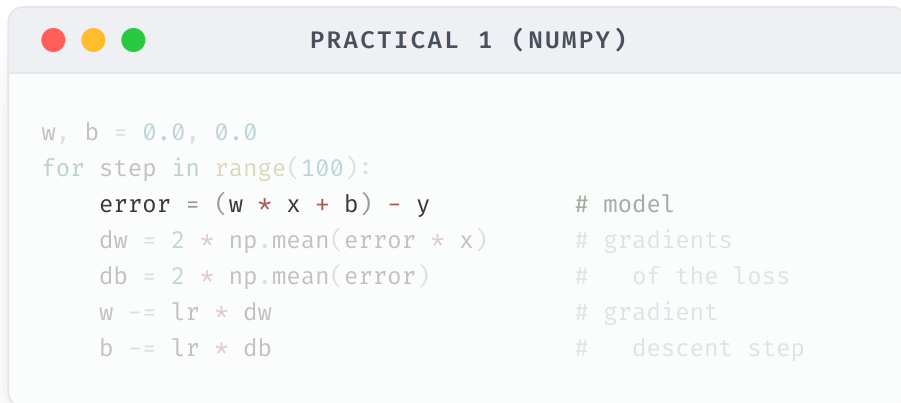
Gradients (how the loss changes with each parameter):

$$\frac{\partial L}{\partial w} = \frac{2}{n} \sum_i (\hat{y}_i - y_i) x_i$$

$$\frac{\partial L}{\partial b} = \frac{2}{n} \sum_i (\hat{y}_i - y_i)$$

Optimiser (gradient descent, learning rate η):

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$



```
w, b = 0.0, 0.0
for step in range(100):
    error = (w * x + b) - y           # model
    dw = 2 * np.mean(error * x)      # gradients
    db = 2 * np.mean(error)           # of the loss
    w -= lr * dw                      # gradient
    b -= lr * db                      # descent step
```

Linear regression has a closed-form solution. We use gradient descent anyway, because it is how models with millions of parameters are trained.

The recipe in miniature: linear regression

Model: $\hat{y} = wx + b$ **Loss** (mean squared error):

$$L(w, b) = \frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$$

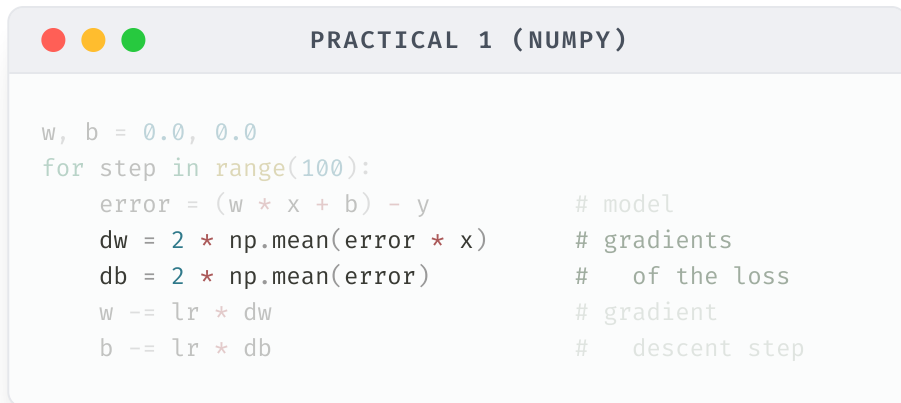
Gradients (how the loss changes with each parameter):

$$\frac{\partial L}{\partial w} = \frac{2}{n} \sum_i (\hat{y}_i - y_i) x_i$$

$$\frac{\partial L}{\partial b} = \frac{2}{n} \sum_i (\hat{y}_i - y_i)$$

Optimiser (gradient descent, learning rate η):

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$



```
w, b = 0.0, 0.0
for step in range(100):
    error = (w * x + b) - y           # model
    dw = 2 * np.mean(error * x)      # gradients
    db = 2 * np.mean(error)           # of the loss
    w -= lr * dw                      # gradient
    b -= lr * db                      # descent step
```

Linear regression has a closed-form solution. We use gradient descent anyway, because it is how models with millions of parameters are trained.

The recipe in miniature: linear regression

Model: $\hat{y} = wx + b$ **Loss** (mean squared error):

$$L(w, b) = \frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$$

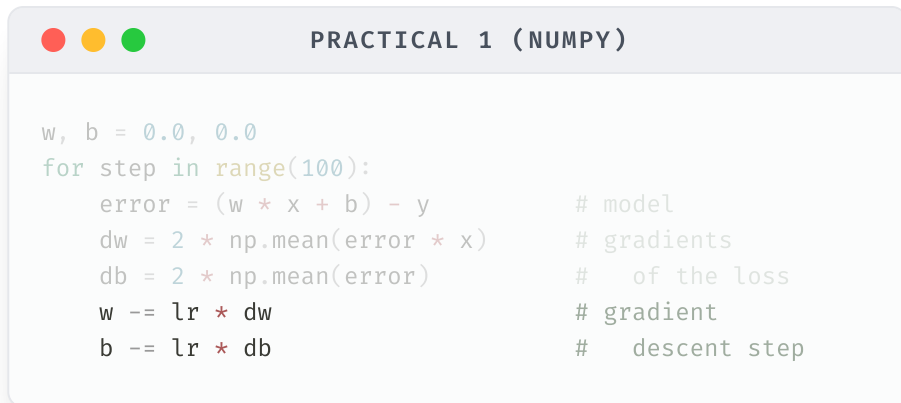
Gradients (how the loss changes with each parameter):

$$\frac{\partial L}{\partial w} = \frac{2}{n} \sum_i (\hat{y}_i - y_i) x_i$$

$$\frac{\partial L}{\partial b} = \frac{2}{n} \sum_i (\hat{y}_i - y_i)$$

Optimiser (gradient descent, learning rate η):

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$



```
w, b = 0.0, 0.0
for step in range(100):
    error = (w * x + b) - y           # model
    dw = 2 * np.mean(error * x)      # gradients
    db = 2 * np.mean(error)          # of the loss
    w -= lr * dw                     # gradient
    b -= lr * db                     # descent step
```

Linear regression has a closed-form solution. We use gradient descent anyway, because it is how models with millions of parameters are trained.

The recipe in miniature: linear regression

Model: $\hat{y} = wx + b$ **Loss** (mean squared error):

$$L(w, b) = \frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$$

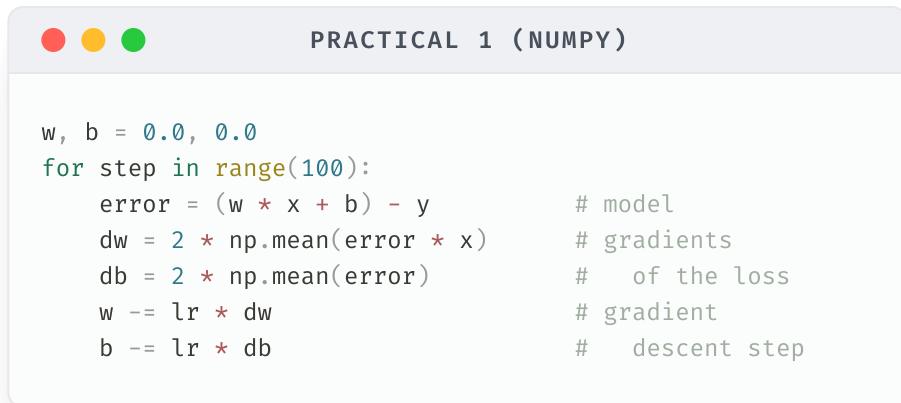
Gradients (how the loss changes with each parameter):

$$\frac{\partial L}{\partial w} = \frac{2}{n} \sum_i (\hat{y}_i - y_i) x_i$$

$$\frac{\partial L}{\partial b} = \frac{2}{n} \sum_i (\hat{y}_i - y_i)$$

Optimiser (gradient descent, learning rate η):

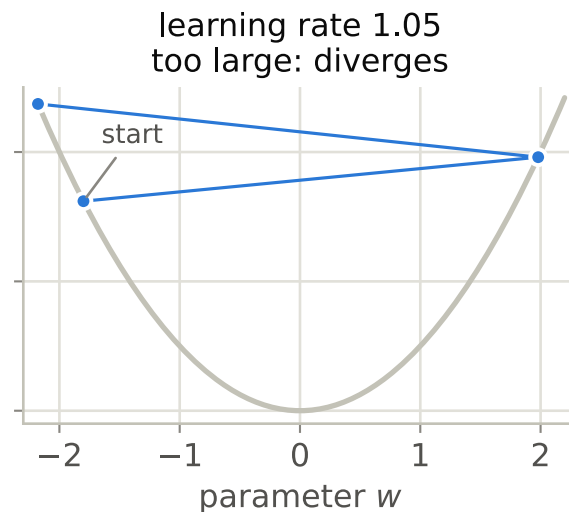
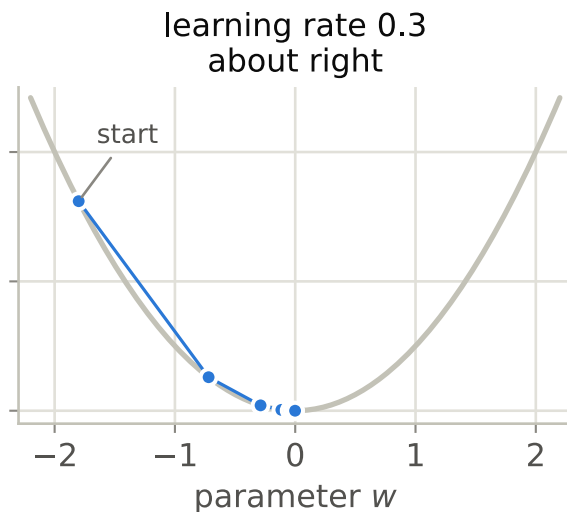
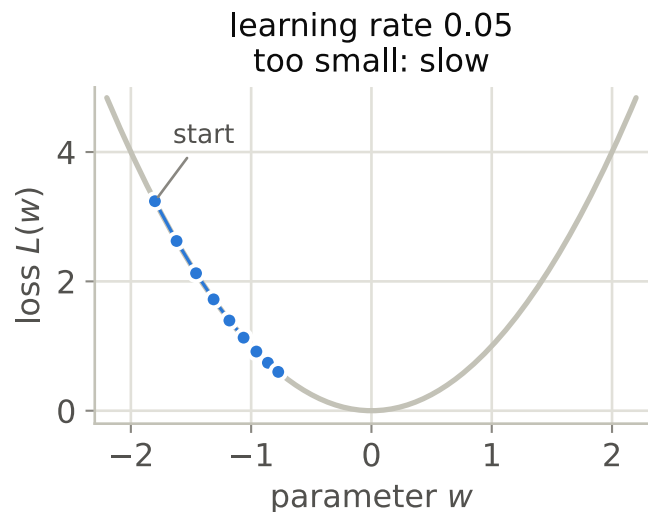
$$w \leftarrow w - \eta \frac{\partial L}{\partial w}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$



```
w, b = 0.0, 0.0
for step in range(100):
    error = (w * x + b) - y           # model
    dw = 2 * np.mean(error * x)       # gradients
    db = 2 * np.mean(error)           # of the loss
    w -= lr * dw                      # gradient
    b -= lr * db                      # descent step
```

Linear regression has a closed-form solution. We use gradient descent anyway, because it is how models with millions of parameters are trained.

Gradient descent: walk downhill

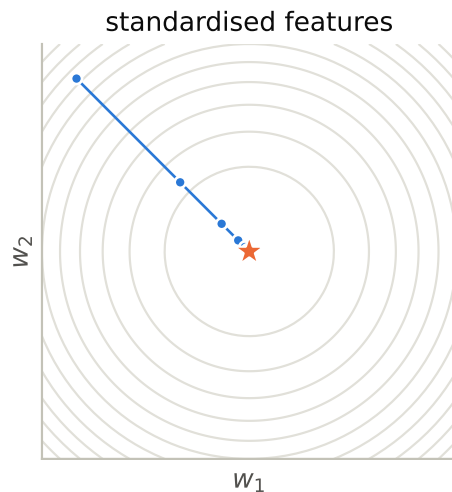
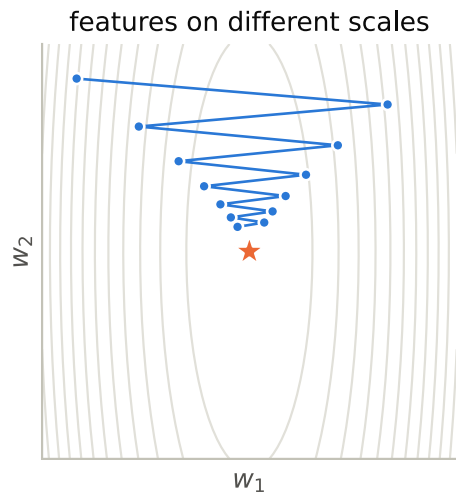


$$\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta)$$

The **learning rate** η is the first hyperparameter everyone tunes: too small wastes compute, too large diverges.

Real training uses **stochastic** (mini-batch) gradient descent: estimate the gradient on a random **batch** of samples per step. One **epoch** is one pass over the whole dataset. Deep-learning training works exactly this way.

Scale your features



When inputs have very different scales (a year around 2000 next to a value around 1; proline in the thousands next to hue around 1), the loss surface becomes a **narrow valley** and gradient descent zig-zags or diverges.

Standardise each feature:

$$x' = \frac{x - \mu}{\sigma}$$

with μ and σ computed on the **training data only**.

Tree-based models do not need scaling; distance-based models (kNN, SVMs) and neural networks do.

The real goal: generalisation

A model that is perfect on its training data can be useless. What matters is performance on **new** data.

Training set (about 70 %)

Validation

Test

Training set: fit the model's **parameters**.

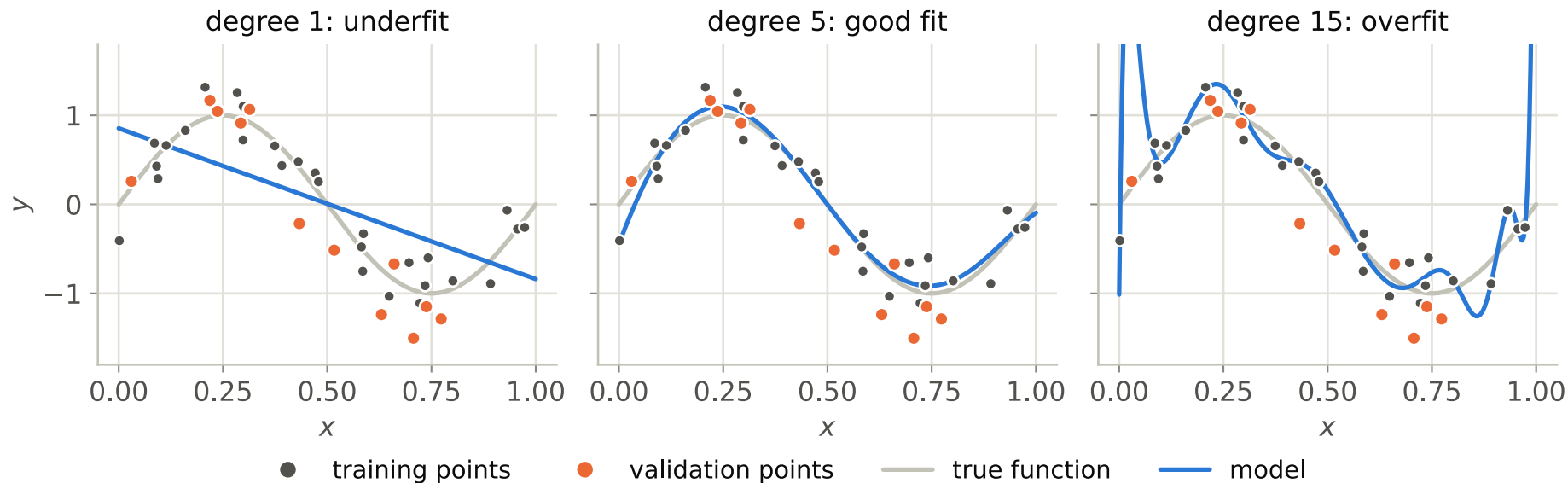
Validation set: choose between models and set **hyperparameters** (learning rate, polynomial degree, ...).

Test set: the final, honest estimate. Used **once**, at the very end.

Split the way the model will be used

Forecasting? Split **by time**. New users, machines or sites? Split **by group**, so data from the same user or machine never appears on both sides.

Underfitting and overfitting

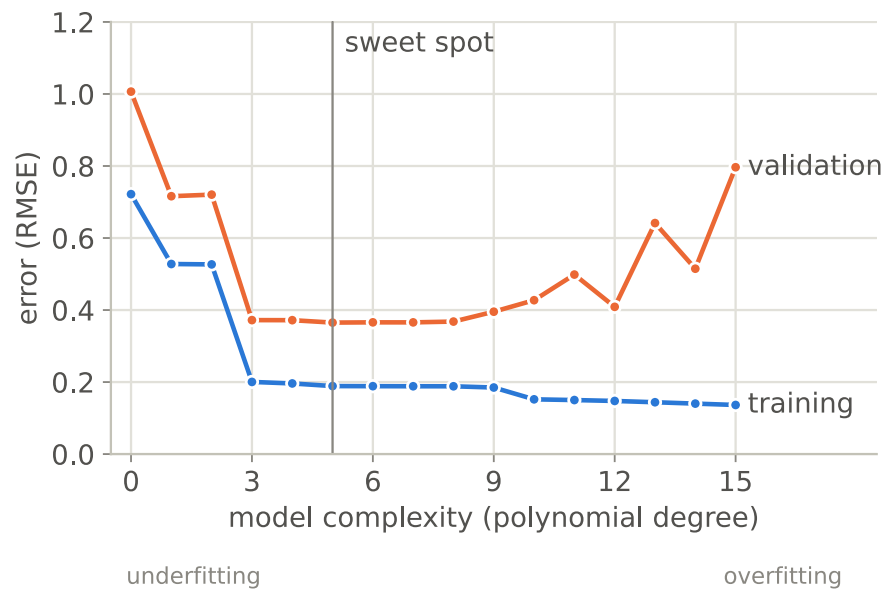


Underfitting (high bias): the model is too simple to capture the pattern. Bad on training *and* new data.

Good fit: captures the signal, ignores the noise.

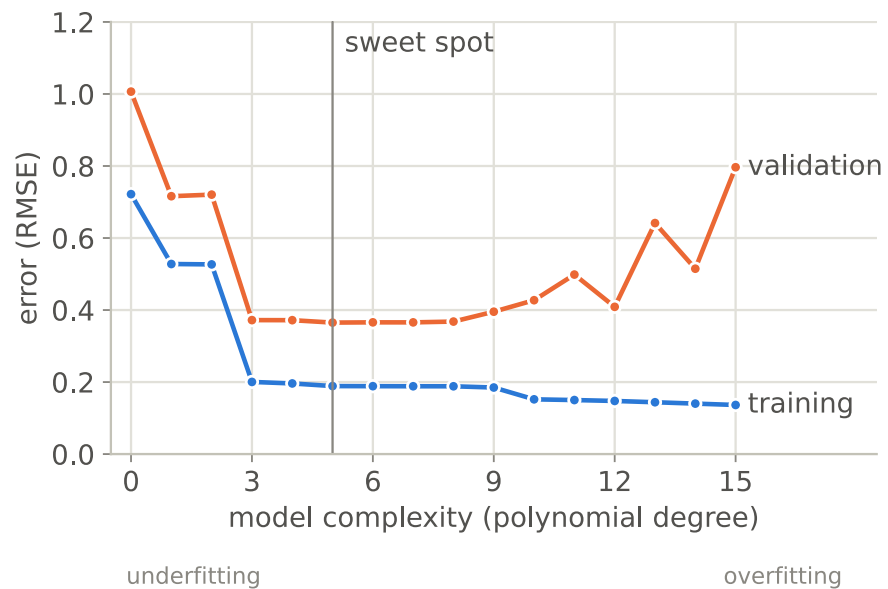
Overfitting (high variance): the model memorises the noise. Excellent on training data, poor on new data.

How to fight overfitting



Training error always falls as complexity grows. **Validation error** tells you where to stop.

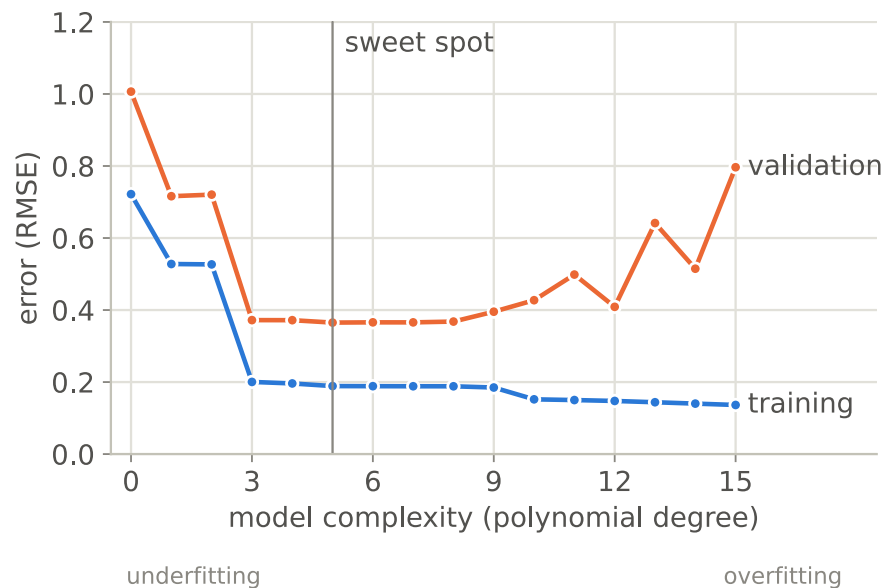
How to fight overfitting



- **More data** (or data augmentation)

Training error always falls as complexity grows. **Validation error** tells you where to stop.

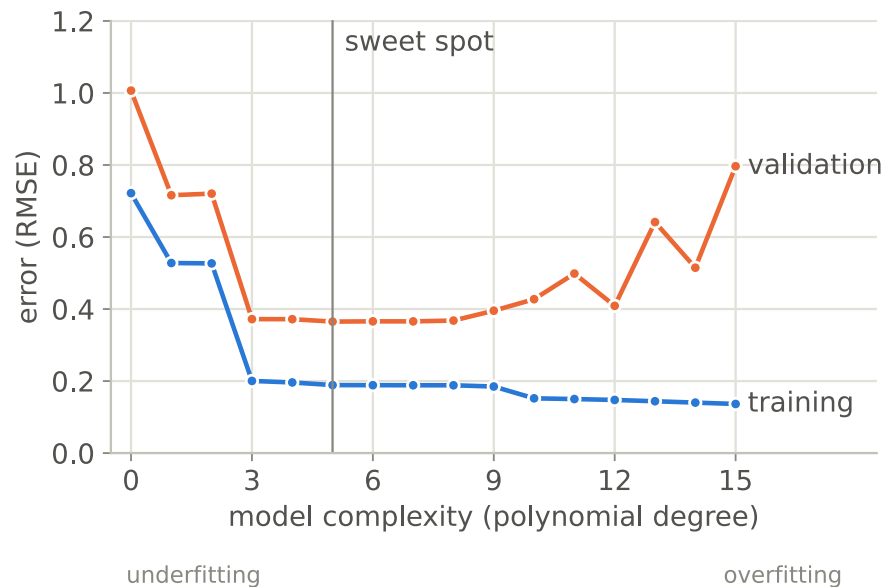
How to fight overfitting



- **More data** (or data augmentation)
- **A simpler model**, or fewer features

Training error always falls as complexity grows. **Validation error** tells you where to stop.

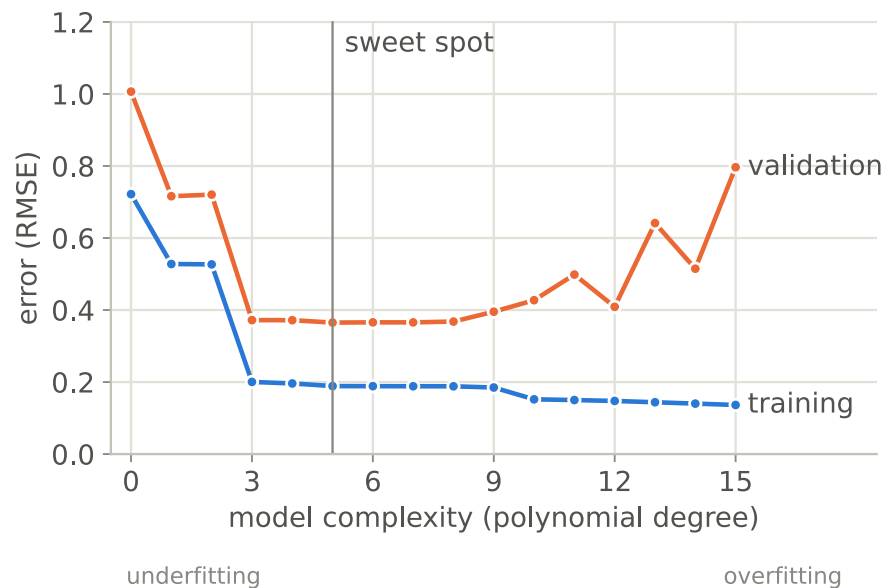
How to fight overfitting



- **More data** (or data augmentation)
- **A simpler model**, or fewer features
- **Regularisation**: penalise large parameters, e.g. ridge $L + \lambda \|\theta\|_2^2$ or lasso $L + \lambda \|\theta\|_1$

Training error always falls as complexity grows. **Validation error** tells you where to stop.

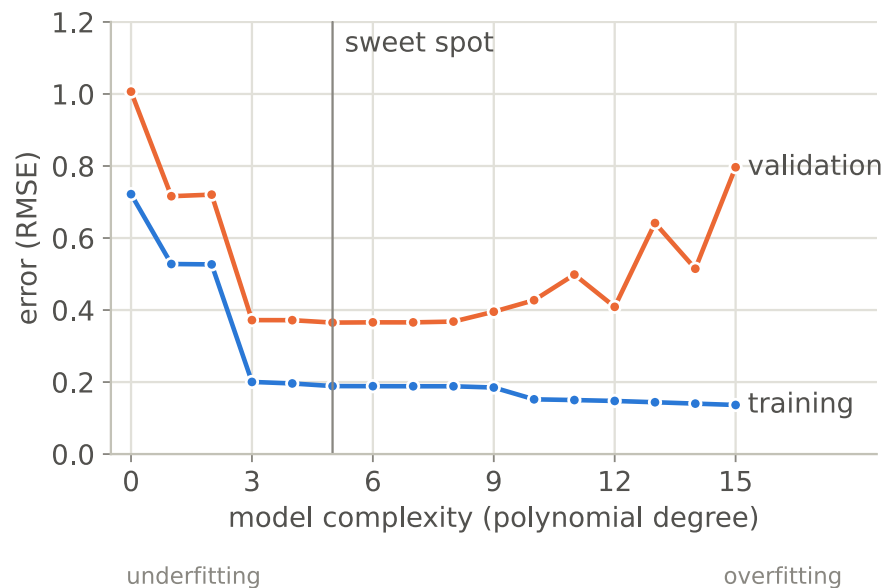
How to fight overfitting



Training error always falls as complexity grows. **Validation error** tells you where to stop.

- **More data** (or data augmentation)
- **A simpler model**, or fewer features
- **Regularisation**: penalise large parameters, e.g. ridge $L + \lambda \|\theta\|_2^2$ or lasso $L + \lambda \|\theta\|_1$
- **Early stopping**: stop training when validation error starts rising

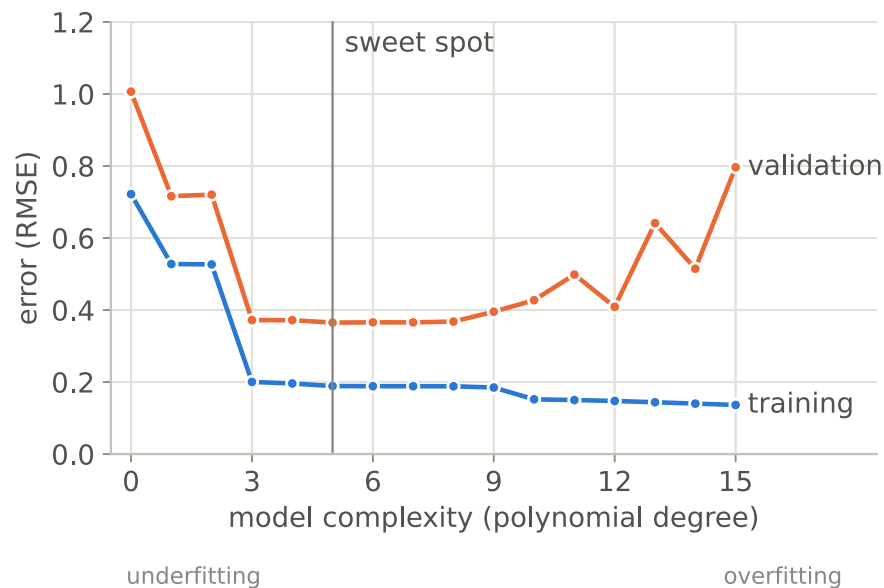
How to fight overfitting



Training error always falls as complexity grows. **Validation error** tells you where to stop.

- **More data** (or data augmentation)
- **A simpler model**, or fewer features
- **Regularisation**: penalise large parameters, e.g. ridge $L + \lambda \|\theta\|_2^2$ or lasso $L + \lambda \|\theta\|_1$
- **Early stopping**: stop training when validation error starts rising
- **Ensembles**: average many models

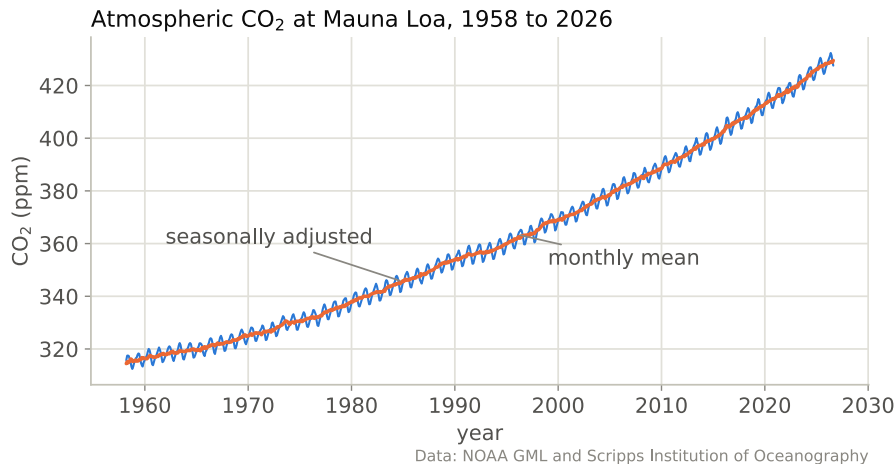
How to fight overfitting



Training error always falls as complexity grows. **Validation error** tells you where to stop.

- **More data** (or data augmentation)
- **A simpler model**, or fewer features
- **Regularisation**: penalise large parameters, e.g. ridge $L + \lambda \|\theta\|_2^2$ or lasso $L + \lambda \|\theta\|_1$
- **Early stopping**: stop training when validation error starts rising
- **Ensembles**: average many models
- **Prior knowledge**: features and constraints you already know are true

Practical 1: the Keeling curve



Connect to Jupyter

Open OnDemand → **My Interactive Sessions** →
Connect to Jupyter, then open the
`intro_to_ai_ml` folder.

Notebook: `01_data_and_learning.ipynb` · 25 min

1. Load and inspect the data with **pandas**
2. Find the **data-quality trap**
3. Visualise; compute growth rates per decade
4. Code **gradient descent** by hand; break it with bad learning rates and unscaled inputs
5. Test on the **future**: train to 2005, predict 2006 onwards
6. *Bonus*: feature engineering and a forecast

Solutions are hidden under **Show solution** after each exercise. Try first!

Part 2

Classical machine learning with scikit-learn

The scientific Python stack

NumPy: n-dimensional arrays, vectorised maths, linear algebra. Everything else builds on it.

pandas: tables with named columns: reading files, cleaning, grouping, joining.

Matplotlib: plots and figures for exploration and for papers.

SciPy: optimisation, statistics, signal processing, sparse matrices.

scikit-learn: classical ML: preprocessing, models, model selection, metrics. **Today.**

Deep-learning frameworks: build and train neural networks on GPUs. **Next step.**

ON A MELUXINA COMPUTE NODE

```
module load env/release/2025.1
module load scikit-learn/1.7.0-gfbf-2025a matplotlib/3.10.3-gfbf-2025a # brings NumPy, SciPy, pandas
```

One API for every model

SCIKIT-LEARN IN FIVE LINES

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

model = make_pipeline(StandardScaler(), RandomForestClassifier())
model.fit(X_train, y_train)          # learn from training data
y_pred = model.predict(X_test)       # predict new samples
model.score(X_test, y_test)          # accuracy ( $R^2$  for regressors)
```

- `X` has shape `(n_samples, n_features)`, `y` has shape `(n_samples,)`
- **Estimators** learn with `.fit()` and predict with `.predict()`
- **Transformers** (scalers, PCA, ...) use `.fit()` and `.transform()`
- A **pipeline** chains them; fitting it fits every step on the training data only
- Swapping the model is a **one-line change**

One API for every model

SCIKIT-LEARN IN FIVE LINES

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

model = make_pipeline(StandardScaler(), RandomForestClassifier())
model.fit(X_train, y_train)          # learn from training data
y_pred = model.predict(X_test)       # predict new samples
model.score(X_test, y_test)          # accuracy ( $R^2$  for regressors)
```

- `X` has shape `(n_samples, n_features)`, `y` has shape `(n_samples,)`
- **Estimators** learn with `.fit()` and predict with `.predict()`
- **Transformers** (scalers, PCA, ...) use `.fit()` and `.transform()`
- A **pipeline** chains them; fitting it fits every step on the training data only
- Swapping the model is a **one-line change**

One API for every model

SCIKIT-LEARN IN FIVE LINES

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

model = make_pipeline(StandardScaler(), RandomForestClassifier())
model.fit(X_train, y_train)          # learn from training data
y_pred = model.predict(X_test)       # predict new samples
model.score(X_test, y_test)          # accuracy ( $R^2$  for regressors)
```

- `X` has shape `(n_samples, n_features)`, `y` has shape `(n_samples,)`
- **Estimators** learn with `.fit()` and predict with `.predict()`
- **Transformers** (scalers, PCA, ...) use `.fit()` and `.transform()`
- A **pipeline** chains them; fitting it fits every step on the training data only
- Swapping the model is a **one-line change**

One API for every model

SCIKIT-LEARN IN FIVE LINES

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

model = make_pipeline(StandardScaler(), RandomForestClassifier())
model.fit(X_train, y_train)          # learn from training data
y_pred = model.predict(X_test)       # predict new samples
model.score(X_test, y_test)          # accuracy ( $R^2$  for regressors)
```

- `X` has shape `(n_samples, n_features)`, `y` has shape `(n_samples,)`
- **Estimators** learn with `.fit()` and predict with `.predict()`
- **Transformers** (scalers, PCA, ...) use `.fit()` and `.transform()`
- A **pipeline** chains them; fitting it fits every step on the training data only
- Swapping the model is a **one-line change**

One API for every model

SCIKIT-LEARN IN FIVE LINES

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

model = make_pipeline(StandardScaler(), RandomForestClassifier())
model.fit(X_train, y_train)          # learn from training data
y_pred = model.predict(X_test)       # predict new samples
model.score(X_test, y_test)          # accuracy ( $R^2$  for regressors)
```

- `X` has shape `(n_samples, n_features)`, `y` has shape `(n_samples,)`
- **Estimators** learn with `.fit()` and predict with `.predict()`
- **Transformers** (scalers, PCA, ...) use `.fit()` and `.transform()`
- A **pipeline** chains them; fitting it fits every step on the training data only
- Swapping the model is a **one-line change**

One API for every model

SCIKIT-LEARN IN FIVE LINES

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

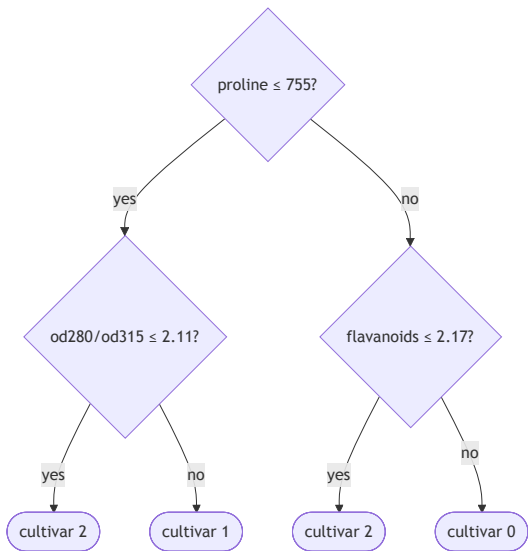
model = make_pipeline(StandardScaler(), RandomForestClassifier())
model.fit(X_train, y_train)          # learn from training data
y_pred = model.predict(X_test)       # predict new samples
model.score(X_test, y_test)          # accuracy ( $R^2$  for regressors)
```

- `X` has shape `(n_samples, n_features)`, `y` has shape `(n_samples,)`
- **Estimators** learn with `.fit()` and predict with `.predict()`
- **Transformers** (scalers, PCA, ...) use `.fit()` and `.transform()`
- A **pipeline** chains them; fitting it fits every step on the training data only
- Swapping the model is a **one-line change**

A field guide to classical models

Model	Good for	Watch out for
Linear / logistic regression (+ ridge, lasso)	baselines, few samples, interpretable coefficients	only linear, unless you engineer features
k-nearest neighbours	simple, no training	needs scaling; weak in high dimensions
Support vector machines	small to medium data, non-linear via kernels	needs scaling; slow beyond $\sim 10^5$ samples
Decision tree	human-readable rules	overfits on its own
Random forest	robust default for tabular data	large models; cannot extrapolate
Gradient boosting (XGBoost, LightGBM, HistGB)	often the most accurate on tabular data	more hyperparameters to tune
Gaussian process regression	small data, uncertainty estimates	cost grows as n^3
k-means, PCA (UMAP, t-SNE for plots)	structure without labels	results depend on scaling and on k

Trees and ensembles



A depth-2 decision tree learned from the wine data (92 % training accuracy)

- A **decision tree** asks a sequence of yes/no questions about single features.
- A **random forest** trains many trees, each on a random resample of the data and random subsets of features, then **averages their votes**. Averaging reduces variance.
- **Gradient boosting** adds small trees **one after another**, each fitted to the errors of the ensemble so far.

On tabular data, tree ensembles are still very hard to beat, even by deep learning (Grinsztajn et al., NeurIPS 2022). Try them before a neural network.

Unsupervised learning: PCA and clustering

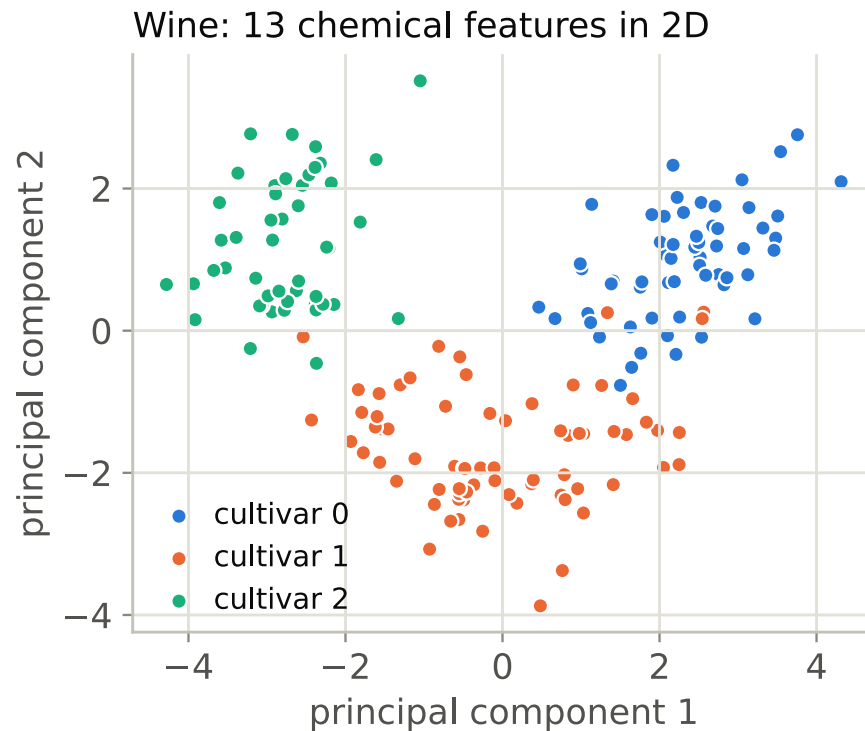
Principal component analysis (PCA)

- Find orthogonal directions of **maximum variance**: the eigenvectors of the covariance matrix
- Project onto the first few to **plot** high-dimensional data or to compress features

k-means clustering

- Partition samples into k groups, minimising
$$\sum_i \|x_i - \mu_{c(i)}\|^2$$
- Choose k with the silhouette score, and **common sense**: do the groups mean something?

Also useful: hierarchical clustering, DBSCAN, Gaussian mixtures.



PCA never saw the labels: colours are added afterwards.

Measuring success: metrics

Regression

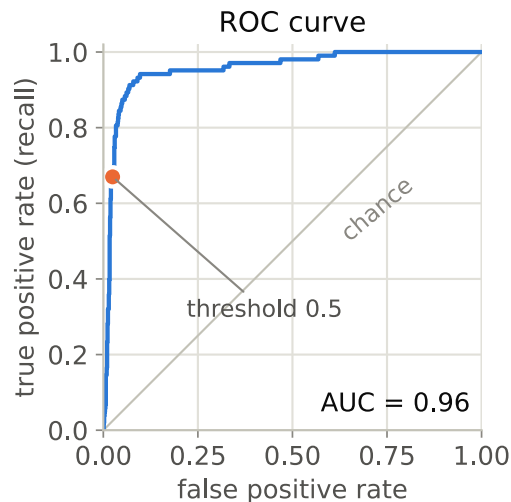
- **MAE**: mean absolute error, in the units of y
- **RMSE**: root mean squared error; punishes large errors
- R^2 : fraction of variance explained (1 is perfect, 0 is "predict the mean")
- Always plot **predicted against true**

Classification

- **Precision** = $\frac{TP}{TP+FP}$: of what I flagged, how much is real?
- **Recall** = $\frac{TP}{TP+FN}$: of what is real, how much did I find?
- **ROC curve**: recall against false alarms for every decision threshold; the area under it (**AUC**) is 1 when perfect, 0.5 for guessing

Confusion matrix

	predicted +	predicted -
actual +	TP 69	FN 34
actual -	FP 22	TN 875

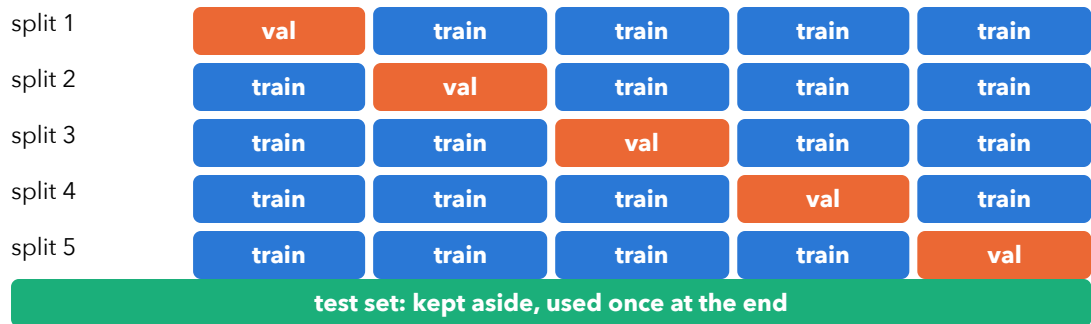


Logistic regression on synthetic data in which 10 % of the samples are positive

High accuracy can be useless

This model is 94 % accurate, yet misses 34 of 103 positives. Always predicting "negative" would already score 90 %. Compare against a **baseline**, and pick metrics that reflect the **cost of each kind of error**.

Cross-validation



- Split the **training** data into k folds; train on $k - 1$, validate on the last; rotate
- Report **mean $\pm$ std** over folds: a fairer comparison than one split
- Tune hyperparameters with `GridSearchCV`
- Keep groups together with `GroupKFold` ; respect time with `TimeSeriesSplit`

Data leakage: the most common mistake

Preprocessing before splitting. Scaling, imputation or **feature selection** fitted on the full dataset lets information from the test set into training.

The same thing on both sides. Repeated measurements of the same thing, the same user or machine, or near-duplicate records in both train and test.

Random splits of structured data. Neighbouring time points or nearby locations are nearly identical: a random split tests interpolation, not prediction.

Features that encode the answer. A machine ID, file name or date that happens to correlate with the label but is not the real cause.

Leakage has been found in **294 papers across 17 fields** (Kapoor & Narayanan, *Patterns* 2023). Fixes: **split first**, keep every learned step **inside a pipeline**, split **by group or time**, and distrust results that look too good.

Practical 2: classifying wines by their chemistry

Notebook: `02_scikit_learn.ipynb` · 20 min

1. Explore the data; make an honest train/test split
2. A **baseline**, then kNN with and without **scaling**
3. **Compare five models** with cross-validation
4. **Tune** an SVM with `GridSearchCV` ; test once
5. **PCA** and **k-means** without labels
6. **Leakage**: 85 % accuracy from pure noise, and how to prevent it

Dataset

178 wines from three grape cultivars, 13 chemical measurements each (Forina et al., UCI). Bundled with scikit-learn: no download.

Fast finishers

Permutation importance, choosing k with the silhouette score, and a medical dataset where accuracy is not enough.

Lunch break

13:00 to 14:00

Back at **14:00**

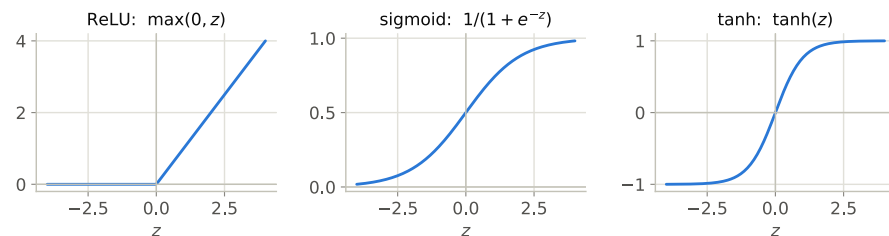
Part 3

Neural networks, transformers and ML on HPC

The artificial neuron

A neuron computes a **weighted sum** of its inputs plus a **bias**, then applies a non-linear **activation function** σ :

$$a = \sigma\left(\sum_j w_j x_j + b\right) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$



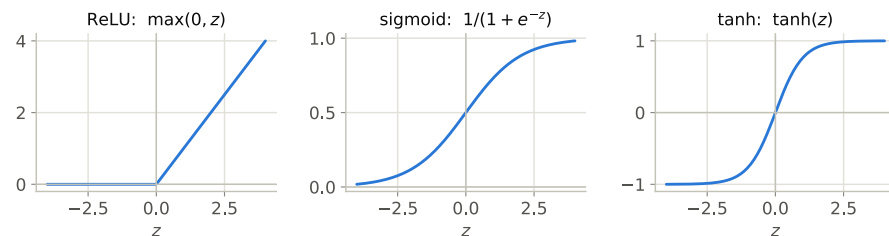
ReLU is the default in modern networks: cheap, and its gradient does not vanish for positive inputs.

The artificial neuron

A neuron computes a **weighted sum** of its inputs plus a **bias**, then applies a non-linear **activation function** σ :

$$a = \sigma\left(\sum_j w_j x_j + b\right) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

- With a sigmoid activation, one neuron **is logistic regression**.



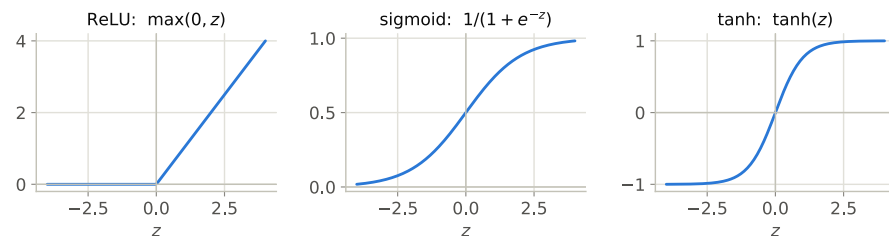
ReLU is the default in modern networks: cheap, and its gradient does not vanish for positive inputs.

The artificial neuron

A neuron computes a **weighted sum** of its inputs plus a **bias**, then applies a non-linear **activation function** σ :

$$a = \sigma\left(\sum_j w_j x_j + b\right) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

- With a sigmoid activation, one neuron **is logistic regression**.
- The weights $\mathbf{w}$ and bias b are the parameters θ that training adjusts.



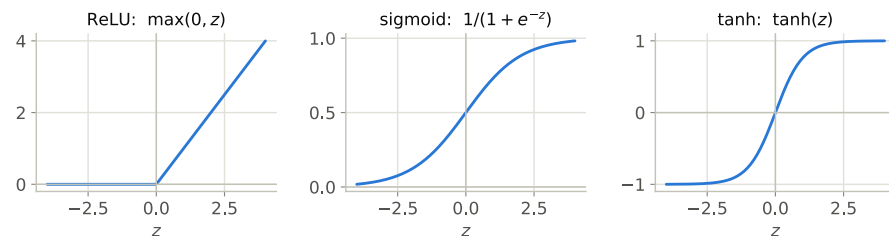
ReLU is the default in modern networks: cheap, and its gradient does not vanish for positive inputs.

The artificial neuron

A neuron computes a **weighted sum** of its inputs plus a **bias**, then applies a non-linear **activation function** σ :

$$a = \sigma\left(\sum_j w_j x_j + b\right) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

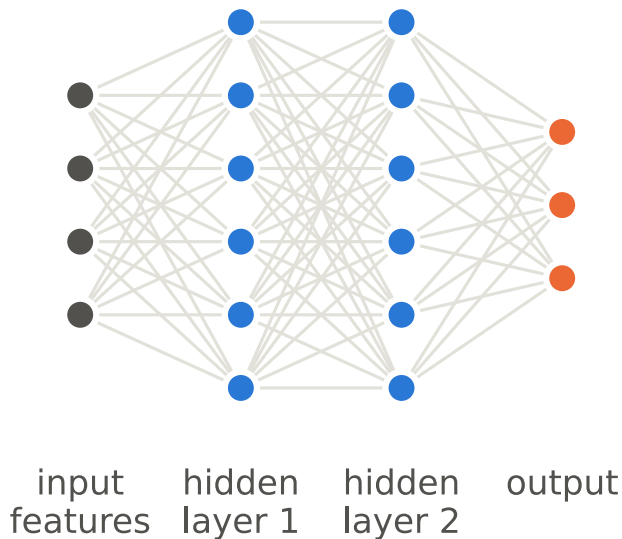
- With a sigmoid activation, one neuron **is logistic regression**.
- The weights $\mathbf{w}$ and bias b are the parameters θ that training adjusts.
- The **non-linearity is essential**: without it, stacked layers collapse into one linear map.



ReLU is the default in modern networks: cheap, and its gradient does not vanish for positive inputs.

Stacking neurons: the multi-layer perceptron

every line is a weight w_{ij} ; every node computes $\sigma(\sum_i w_{ij}x_i + b_j)$



- Neurons are arranged in **layers**; each layer feeds the next
- A layer applied to a batch is a **matrix multiplication** plus an activation:
$$H = \sigma(XW + b)$$
- **Depth** lets the network build features from features
- One hidden layer can approximate any continuous function (**universal approximation**), but depth makes it efficient

Training uses the recipe from Part 1:

backpropagation (the chain rule, layer by layer)
gives the gradient for every weight, and gradient descent updates them, one mini-batch at a time.

Why deep learning needs GPUs

A layer on a batch is a **matrix multiplication**:

$$\underbrace{X}_{\text{batch} \times \text{in}} \underbrace{W}_{\text{in} \times \text{out}}$$

costs about $2 \times \text{batch} \times \text{in} \times \text{out}$ floating-point operations (FLOPs).

Training costs roughly **6 × parameters** FLOPs for every training example seen (forward + backward pass).

GPT-3: $6 \times 175 \text{ billion parameters} \times 300 \text{ billion tokens} \approx 3 \times 10^{23}$ FLOPs.

Deep learning tolerates **low precision** (BF16, FP16, even FP8), unlike most simulation codes. Tensor cores exploit this.

MeluXina node	Compute (theoretical peak)
CPU node: 128 cores	a few TFLOP/s (FP64)
GPU node: 4 × NVIDIA A100 40 GB	4 × 312 TFLOP/s (BF16, tensor cores)

- Thousands of simple cores doing multiply-adds in parallel
- Fast on-board memory (about 1.5 TB/s per A100)
- Nodes linked by fast networks for **multi-GPU training** (DDP)

Match the architecture to the data

Data	Architecture	Built-in assumption	Examples
Tabular	MLP (but try tree ensembles first)	none	job logs, sensor readings, database records
Images and grids	CNN, U-Net, vision transformer	nearby pixels belong together; a pattern means the same anywhere	photo recognition, medical scans, satellite images
Sequences and text	RNN/LSTM, transformer	order; long-range context (attention)	language models, speech, time-series forecasting
Graphs and meshes	graph neural networks	the numbering of the nodes does not matter	computer networks, simulation meshes, molecules
Physical systems	physics-informed NNs, neural operators	known equations as constraints	fast surrogates for simulations

Use what you know

Build what you know about the problem (its **structure and constraints**) into the model. It needs less data and generalises better.

Transformers: the architecture behind LLMs

1. Text becomes vectors. Text is split into **tokens** (words or pieces of words) and each token becomes a vector of numbers.

The job failed because it ran out of memory

2. Self-attention. Each token looks at all the others and weighs how much each one matters: to work out what **it** means, the model attends to **job**. For the token vectors X :

$$\text{Attention} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V, \quad Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

3. Stack blocks. Attention plus a small MLP form one block; GPT-3 stacks 96. The last layer gives a probability for every possible next token.

Vaswani et al., "Attention is all you need", 2017

Transformers: the architecture behind LLMs

1. Text becomes vectors. Text is split into **tokens** (words or pieces of words) and each token becomes a vector of numbers.

The job failed because it ran out of memory

2. Self-attention. Each token looks at all the others and weighs how much each one matters: to work out what **it** means, the model attends to **job**. For the token vectors X :

$$\text{Attention} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V, \quad Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

3. Stack blocks. Attention plus a small MLP form one block; GPT-3 stacks 96. The last layer gives a probability for every possible next token.

- **Training:** predict the next token across trillions of tokens of text. The same recipe (data, model, loss, optimiser), with no labels needed.

Vaswani et al., "Attention is all you need", 2017

Transformers: the architecture behind LLMs

1. Text becomes vectors. Text is split into **tokens** (words or pieces of words) and each token becomes a vector of numbers.

The job failed because it ran out of memory

2. Self-attention. Each token looks at all the others and weighs how much each one matters: to work out what **it** means, the model attends to **job**. For the token vectors X :

$$\text{Attention} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V, \quad Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

3. Stack blocks. Attention plus a small MLP form one block; GPT-3 stacks 96. The last layer gives a probability for every possible next token.

- **Training:** predict the next token across trillions of tokens of text. The same recipe (data, model, loss, optimiser), with no labels needed.
- **Generating:** predict a token, append it, repeat.

Vaswani et al., "Attention is all you need", 2017

Transformers: the architecture behind LLMs

1. Text becomes vectors. Text is split into **tokens** (words or pieces of words) and each token becomes a vector of numbers.

The job failed because it ran out of memory

2. Self-attention. Each token looks at all the others and weighs how much each one matters: to work out what **it** means, the model attends to **job**. For the token vectors X :

$$\text{Attention} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V, \quad Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

3. Stack blocks. Attention plus a small MLP form one block; GPT-3 stacks 96. The last layer gives a probability for every possible next token.

- **Training:** predict the next token across trillions of tokens of text. The same recipe (data, model, loss, optimiser), with no labels needed.
- **Generating:** predict a token, append it, repeat.
- Everything is **matrix multiplication**: ideal for GPUs.

Vaswani et al., "Attention is all you need", 2017

Transformers: the architecture behind LLMs

1. Text becomes vectors. Text is split into **tokens** (words or pieces of words) and each token becomes a vector of numbers.

The job failed because it ran out of memory

2. Self-attention. Each token looks at all the others and weighs how much each one matters: to work out what **it** means, the model attends to **job**. For the token vectors X :

$$\text{Attention} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V, \quad Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

3. Stack blocks. Attention plus a small MLP form one block; GPT-3 stacks 96. The last layer gives a probability for every possible next token.

- **Training:** predict the next token across trillions of tokens of text. The same recipe (data, model, loss, optimiser), with no labels needed.
- **Generating:** predict a token, append it, repeat.
- Everything is **matrix multiplication**: ideal for GPUs.
- **Cost:** attention compares every pair of tokens, so it grows with the **square** of the context length.

Vaswani et al., "Attention is all you need", 2017

Transformers: the architecture behind LLMs

1. Text becomes vectors. Text is split into **tokens** (words or pieces of words) and each token becomes a vector of numbers.

The job failed because it ran out of memory

2. Self-attention. Each token looks at all the others and weighs how much each one matters: to work out what **it** means, the model attends to **job**. For the token vectors X :

$$\text{Attention} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V, \quad Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

3. Stack blocks. Attention plus a small MLP form one block; GPT-3 stacks 96. The last layer gives a probability for every possible next token.

- **Training:** predict the next token across trillions of tokens of text. The same recipe (data, model, loss, optimiser), with no labels needed.
- **Generating:** predict a token, append it, repeat.
- Everything is **matrix multiplication**: ideal for GPUs.
- **Cost:** attention compares every pair of tokens, so it grows with the **square** of the context length.
- **Inference** keeps the keys and values of past tokens in GPU memory (the **KV cache**): memory, not FLOPs, often sets the limit.

Vaswani et al., "Attention is all you need", 2017

Foundation models and LLMs

Foundation models are very large networks, **pretrained** with self-supervision on broad data, then **adapted** to many tasks by fine-tuning or prompting.

- **Language and code:** GPT, Llama, Claude, Mistral
- **Images:** models such as CLIP that link images and text
- **Science:** models for proteins, weather and satellite data

In your work you can use them as **assistants** (coding, documentation, literature), or as **feature extractors**: embeddings from a pretrained model, fed into a small model trained on your data.

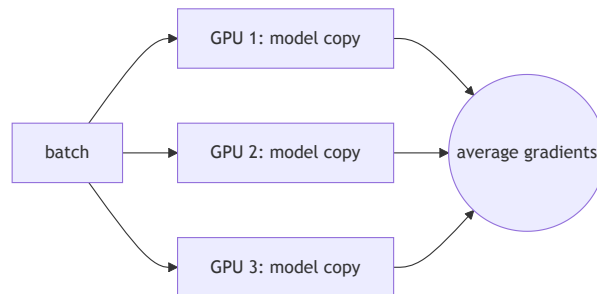
Use with care

- They can be **confidently wrong**: verify code, facts and citations
- Do not paste **unpublished or personal data** into external services
- Model versions change: record which one you used
- Training them costs enormous compute; **fine-tuning or reusing** one is far cheaper

When do you need a supercomputer for ML?

Situation	Where to run it
Small tabular data, scikit-learn	a laptop, or a few CPU cores
Many independent fits: hyperparameter searches, cross-validation, ensembles	many CPU cores , or a Slurm job array
Deep learning on images, 3D volumes, long sequences, large datasets	GPU nodes
Models or datasets too large or too slow for one GPU	multi-GPU, multi-node training (DDP)
Applying a trained model to millions of samples (inference)	GPU or CPU job arrays

Data parallelism (DDP): every GPU holds a copy of the model, processes a different slice of each batch, and the gradients are **averaged across GPUs** before every update.



Good habits for ML on MeluXina

Environment

- Use **modules**, **Apptainer containers** or virtual environments; build them on compute nodes
- **Never** train on the login node; debug with a small subset on `qos=test`, then scale up

Data

- Millions of small files overload parallel file systems: pack them into large files (HDF5, Zarr, Parquet, WebDataset)
- Use data-loader workers so the GPU never waits for data

Robustness

- Jobs hit walltime limits: **checkpoint** model and optimiser state regularly and make jobs resumable
- Watch GPU utilisation (`nvidia-smi`): an idle GPU is wasted allocation and energy

Reproducibility

- Fix **random seeds**; log hyperparameters and software versions
- Commit code with **Git** and store the commit hash with every result

The energy cost of AI

- Training GPT-3 was estimated at about **1,300 MWh** and **550 t CO₂e** (Patterson et al., 2021)
- Over a popular model's lifetime, **inference** can use more energy than training
- Your own hyperparameter sweeps add up: 100 configurations × 10 GPU-hours = 1,000 GPU-hours

AI for decarbonisation

ML also helps cut emissions: forecasting wind and solar output, balancing power grids and cooling data centres more efficiently.

What you can do

- **Start small**; scale up only when the small version works
- **Reuse** pretrained models; fine-tune instead of training from scratch
- Search hyperparameters **smartly** (random or Bayesian search, early stopping) rather than exhaustive grids
- Use **mixed precision** (BF16) and keep GPUs busy
- **Measure**: job accounting, tools such as CodeCarbon, and report the compute you used

Practical 3: a first neural network



FIRST, IN A MELUXINA TERMINAL

```
cd day_2/notebooks/intro_to_ai_ml
sbatch train_digits.slurm      # runs while you work
```

Then notebook: `03_neural_networks.ipynb` · 25 min

1. 8×8 handwritten digits as feature vectors
2. Train an **MLP**; read its training curve
3. Is it better than classical ML?
4. Rebuild its predictions with **plain NumPy matrix multiplications**
5. Collect your **batch job**: `cat digits-<jobid>.out`

The batch job

A hyperparameter search (24 configurations $\times$ 5 folds), run on **1 core** and then on **16 cores**. What speed-up do you get? Why is it less than $16\times$? What does starting the workers cost?

Under the hood

Section 4 shows everything a trained network computes: matrix multiplications and a simple activation. Larger networks do the same, on GPUs.

Checklist: machine learning in practice

Checklist: machine learning in practice

1. Start from the **question** and a **baseline**

Checklist: machine learning in practice

1. Start from the **question** and a **baseline**
2. **Look at your data**: ranges, sentinels, gaps,
provenance

Checklist: machine learning in practice

1. Start from the **question** and a **baseline**
2. **Look at your data**: ranges, sentinels, gaps, provenance
3. **Split first**, and split the way the model will be used (by time, by group)

Checklist: machine learning in practice

1. Start from the **question** and a **baseline**
2. **Look at your data**: ranges, sentinels, gaps, provenance
3. **Split first**, and split the way the model will be used (by time, by group)
4. Keep every learned step **inside a pipeline**

Checklist: machine learning in practice

1. Start from the **question** and a **baseline**
2. **Look at your data**: ranges, sentinels, gaps, provenance
3. **Split first**, and split the way the model will be used (by time, by group)
4. Keep every learned step **inside a pipeline**
5. Choose **metrics** that reflect the cost of errors in your application

Checklist: machine learning in practice

1. Start from the **question** and a **baseline**
2. **Look at your data**: ranges, sentinels, gaps, provenance
3. **Split first**, and split the way the model will be used (by time, by group)
4. Keep every learned step **inside a pipeline**
5. Choose **metrics** that reflect the cost of errors in your application
6. Try **simple models** (linear, trees) before deep learning

Checklist: machine learning in practice

1. Start from the **question** and a **baseline**
2. **Look at your data**: ranges, sentinels, gaps, provenance
3. **Split first**, and split the way the model will be used (by time, by group)
4. Keep every learned step **inside a pipeline**
5. Choose **metrics** that reflect the cost of errors in your application
6. Try **simple models** (linear, trees) before deep learning
7. Beware **extrapolation**; quantify uncertainty where you can

Checklist: machine learning in practice

1. Start from the **question** and a **baseline**
2. **Look at your data**: ranges, sentinels, gaps, provenance
3. **Split first**, and split the way the model will be used (by time, by group)
4. Keep every learned step **inside a pipeline**
5. Choose **metrics** that reflect the cost of errors in your application
6. Try **simple models** (linear, trees) before deep learning
7. Beware **extrapolation**; quantify uncertainty where you can
8. Make it **reproducible**: Git, environments, seeds

Checklist: machine learning in practice

1. Start from the **question** and a **baseline**
2. **Look at your data**: ranges, sentinels, gaps, provenance
3. **Split first**, and split the way the model will be used (by time, by group)
4. Keep every learned step **inside a pipeline**
5. Choose **metrics** that reflect the cost of errors in your application
6. Try **simple models** (linear, trees) before deep learning
7. Beware **extrapolation**; quantify uncertainty where you can
8. Make it **reproducible**: Git, environments, seeds
9. Mind the **compute and energy** you use

Where to go next

Documentation

- scikit-learn user guide: scikit-learn.org
- MeluXina documentation: docs.lxp.lu

Reading

- A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, O'Reilly
- S. Prince, *Understanding Deep Learning*, MIT Press 2023, free at udlbook.github.io
- C. Bishop & H. Bishop, *Deep Learning: Foundations and Concepts*, Springer 2024
- S. Kapoor et al., *REFORMS: consensus-based recommendations for machine-learning-based science*, Science Advances 2024

Thank you

Questions?

Slides and notebooks: git.ichec.ie/decarb-ai-2025/oct_26